T R A N S G U A R D

MEMORIA TÉCNICA DEL PROYECTO DE ESCRITORIO

TransGuard

Sistema de apoyo a la operación de una terminal portuaria, con marco PEAS, lógica proposicional y tablas de verdad.

TECNOLOGÍAS	Python · Tkinter · MongoDB · Ollama (llama3)
COMPONENTES	Tutor de IA (ejercicio 1) · Panel integral TransGuard (ejercicio 2)
ALCANCE	Marco PEAS, lógica proposicional, tablas de verdad, clasificación híbrida, RAG y riesgos éticos

Contenido

1. Marco PEAS 4

1.1 PEAS de TransGuard (terminal portuaria) 4

1.2 Propiedades del entorno y tipo de agente 5

1.3 PEAS por módulo 5

2. Lógica proposicional 6

2.1 Proposiciones atómicas del dominio portuario 6

2.2 Conectivos lógicos 6

2.3 Reglas de decisión como fórmulas 6

2.4 Equivalencias utilizadas 7

2.5 Inferencia: de las premisas a la decisión 7

2.6 Clasificación de las fórmulas 7

3. Tablas de verdad 8

3.1 Conectivos básicos 8

3.2 Tablas reducidas por regla 8

3.3 Ejemplos de lectura 8

3.4 Tabla de verdad completa (32 combinaciones) 9

3.5 Conteos, solapamientos y distribución de decisiones 9

4. Ficha del proyecto 11

5. Puesta en marcha 11

5.1 Requisitos 11

5.2 Ejecución 11

5.3 Variables de entorno opcionales 11

6. Diseño interno 11

Archivos 12

7. Datos que guarda la terminal 12

8. Motor de decisiones (lógica proposicional) 13

8.1 Premisas del dominio portuario 13

8.2 Reglas 13

8.3 Tablas de verdad de las reglas nuevas 13

8.4 Trazabilidad y análisis 13

9. Clasificador mixto de incidentes 14

9.1 Categorías del dominio . . . 14
9.2 Flujo . . . 14
9.3 Prompt del clasificador (texto íntegro) . . . 14
9.4 Experimento sobre el dataset (32 correos etiquetados a mano) . . . 14

10. Asistente consultivo (RAG) . . . 15

Prompt del asistente (texto íntegro) . . . 15

11. Registro de riesgos éticos . . . 15

12. Pantalla de escritorio (ejercicio 2) . . . 16

System prompt del tutor (texto íntegro) . . . 17

13. Cómo se comprobó . . . 17

14. Limitaciones conocidas . . . 17

15. Solución rápida de problemas . . . 18

1. Marco PEAS

El marco PEAS (*Performance, Environment, Actuators, Sensors*) describe a un agente racional con cuatro elementos: la **medida de desempeño** con la que se evalúa, el **entorno** en el que actúa, los **actuadores** con los que modifica ese entorno y los **sensores** con los que lo percibe. Formalizar TransGuard de esta manera deja explícito qué se espera del sistema, en qué condiciones trabaja y con qué medios cuenta, antes de entrar al detalle técnico.

1.1 PEAS de TransGuard (terminal portuaria)

Elemento	Descripción aplicada a TransGuard
P · Medida de desempeño	• Decisiones de acceso correctas según las reglas Y, Z, E y A con su precedencia: ninguna unidad sin autorización con vía de ingreso y las emergencias siempre bloqueadas. • Exactitud del clasificador de incidentes sobre el dataset de 32 correos etiquetados a mano (en modo reglas: 90.6 % en categoría y 75 % en prioridad). • Salidas del LLM válidas frente al esquema pydantic y frecuencia de uso del respaldo por reglas. • Trazabilidad: explicación paso a paso guardada con cada acceso. • Respuestas del asistente con cita [coleccion#id] y la respuesta fija “No tengo información registrada sobre eso.” cuando no hay registros. • Reducción del riesgo ético residual frente al inherente (matriz 5×5). • Latencia (reglas < 1 ms; llama3 ~90 s por correo en CPU) y 29/29 pruebas unitarias en verde.
E · Entorno	• Terminal portuaria: unidades TRK-###, muelles, andenes, dársenas, grúas, básculas y TOS. • Documentación portuaria (BL, manifiesto) y mercancías, algunas peligrosas. • Correos de incidentes con redacción variable, incluso con ortografía informal. • MongoDB (`transguard`, 8 colecciones) o respaldo en JSON local. • Servicio local Ollama con el modelo llama3. • Operadores humanos y vigilante de turno que supervisan las decisiones.
A · Actuadores	• Registro en la bitácora de accesos: salidas A, E, Z, Y, decisión, semáforo y explicación. • Dirección de la unidad hacia ingreso libre, canal de inspección, retención documental, protocolo de emergencia o denegación. • Clasificación del incidente: categoría, prioridad, entidades extraídas y bandera `requiere_revision_humana`; cambio de estado (nuevo / en_atencion / cerrado) e historial. • Respuestas del asistente con fuentes citadas. • Escritura en MongoDB o en `local_fallback.json`; guardado de evaluaciones. • KPIs y gráficas del panel; exportación CSV / JSON / PDF.
S · Sensores	• Casillas de premisas P, Q, R, S y D en la pestaña Acceso, y búsqueda por placa o TRK. • Texto de los correos (remitente, asunto, cuerpo) pegados en Incidentes. • Registros de la base de datos que alimentan el asistente (camiones, accesos, incidentes, riesgos_eticos). • Respuesta JSON de Ollama y su validación con pydantic. • Dataset de 32 correos etiquetados para la evaluación. • Estado de MongoDB y del LLM en la cabecera; parámetros de Configuración.

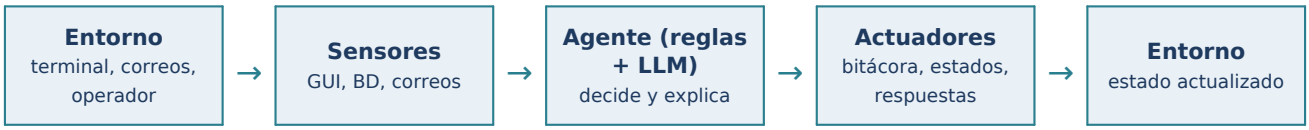
1.2 Propiedades del entorno y tipo de agente

Propiedad	Clasificación	Justificación
Observabilidad	Parcialmente observable	El agente solo conoce las premisas que el operador captura y los registros de la base; un correo puede omitir placa, peso o ubicación (se usa `null`).
Determinismo	Mixto	El motor de reglas es determinista; el LLM y la conducta humana introducen incertidumbre.
Episodios	Episódico	Cada ingreso o incidente se decide por separado; la bitácora sirve para auditoría y para el RAG.
Dinamismo	Semidinámico	Los datos cambian entre decisiones (por ejemplo, `certificacion_dias_restantes`), no mientras se evalúa una.
Discreto / continuo	Discreto	Premisas booleanas, categorías y prioridades en vocabularios finitos.
Número de agentes	Multiagente cooperativo	Interactúan el motor de reglas, el LLM, el operador y el vigilante de turno.

Tipo de agente. El motor de acceso es un *agente reactivo basado en reglas*: asocia cada combinación de percepciones (P, Q, R, S, D) con una acción. El clasificador de incidentes es un *agente híbrido*: combina reglas con un LLM y cede la decisión a un humano cuando ambos discrepan. El asistente es un *agente basado en conocimiento* que solo responde con lo que recupera de la base.

1.3 PEAS por módulo

Módulo	Desempeño	Entorno	Actuadores	Sensores
Motor de acceso	Decisión correcta y explicada	Unidades en ingreso, documentación	Decisión, semáforo, bitácora	Premisas P, Q, R, S, D
Clasificador mixto	Exactitud de categoría y prioridad	Correos de incidentes	Clasificación, entidades, bandera de revisión	Texto del correo, respuesta JSON del LLM
Asistente RAG	Respuestas citadas, sin invenciones	Base de datos y operador	Respuesta con fuentes	Pregunta y registros recuperados
Tutor de IA (ej. 1)	Claridad y utilidad didáctica	Conversación con un alumno	Respuesta y resumen del historial	Mensajes del alumno e historial



2. Lógica proposicional

La lógica proposicional trabaja con **proposiciones**: enunciados que son verdaderos (V) o falsos (F), nunca ambas cosas. Se combinan con **conectivos** para formar fórmulas cuyo valor depende solo de los valores de sus componentes. Así, la política de acceso de la terminal se convierte en un cálculo que se puede probar, explicar y revisar.

2.1 Proposiciones atómicas del dominio portuario

Símbolo	Proposición	Cómo se captura
P	Unidad con autorización previa de ingreso	Casilla en la pestaña Acceso
Q	El peso rebasa el límite de la terminal	Casilla en la pestaña Acceso
R	Mercancía catalogada como peligrosa	Casilla en la pestaña Acceso
S	Operador con licencia/certificado vigente	Casilla en la pestaña Acceso
D	Documentación portuaria completa (BL, manifiesto)	Casilla en la pestaña Acceso

2.2 Conectivos lógicos

Conectivo	Símbolo	Se lee	Verdadero cuando…
Negación	¬p	no p	p es falsa
Conjunción	p ∧ q	p y q	ambas son verdaderas
Disyunción	p ∨ q	p o q (inclusiva)	al menos una es verdadera
Condicional	p → q	si p entonces q	no ocurre p verdadera con q falsa
Bicondicional	p ↔ q	p si y solo si q	ambas tienen el mismo valor

2.3 Reglas de decisión como fórmulas

Regla	Fórmula	Efecto
A	A = P ∧ S ∧ ¬Q	Ingreso libre
E	E = P ∧ (R ∨ Q)	Canal de inspección
Z (nueva)	Z = P ∧ ¬D	Retención documental: la unidad autorizada sin BL o manifiesto no puede desconsolidar
Y (nueva)	Y = P ∧ Q ∧ R	Protocolo de emergencia: sobrepeso y mercancía peligrosa juntos bloquean el acceso hasta permiso especial

Varias reglas pueden cumplirse a la vez, por eso el sistema las resuelve con una **precedencia**: Y > Z > E > A. Si P es falsa no hay vía de ingreso y la unidad se deniega. En forma de procedimiento:

```
decision(P, Q, R, S, D):
    si no P:   denegado                  # sin autorización no hay vía de ingreso
    si Y:      protocolo de emergencia   # bloquea hasta permiso especial
    si Z:      retención documental
    si E:      canal de inspección
    si A:      ingreso libre
    en otro caso: ninguna regla se activa
```

> **Caso sin regla activa.** Al enumerar las 32 combinaciones aparece una en la que ninguna regla se cumple aunque P sea verdadera: P=V, Q=F, R=F, S=F, D=V (unidad autorizada, sin sobrepeso, sin carga peligrosa, con documentos completos, pero con operador sin certificado vigente). El reporte original no define una salida explícita para ella; aquí se rotula “Sin regla activa” y se entiende que no obtiene ingreso libre. Conviene confirmar en `reglas.py` cómo se resuelve.

2.4 Equivalencias utilizadas

Ley	Equivalencia	Uso en TransGuard
Distributiva	$p \wedge (q \vee r) \equiv (p \wedge q) \vee (p \wedge r)$	$E \equiv (P \wedge R) \vee (P \wedge Q)$: la inspección se activa por carga peligrosa o por sobrepeso.
De Morgan	$\neg(p \vee q) \equiv \neg p \wedge \neg q$	$\neg E \equiv \neg P \vee (\neg R \wedge \neg Q)$: no hay inspección si falta autorización o no hay ni peligro ni sobrepeso.
De Morgan	$\neg(p \wedge q) \equiv \neg p \vee \neg q$	$\neg Y \equiv \neg P \vee \neg Q \vee \neg R$: no hay emergencia si falta al menos una de las tres condiciones.
Condicional	$p \rightarrow q \equiv \neg p \vee q$	Leer cada regla como “si se cumplen las premisas, entonces se aplica la acción”.
Conmutativa y asociativa	$p \wedge q \equiv q \wedge p$	El orden en que se capturan las premisas no cambia el resultado.

2.5 Inferencia: de las premisas a la decisión

Cada evaluación aplica *modus ponens* (de p y p → q se concluye q) sobre las reglas, y después elige por precedencia:

Hechos	Regla	Conclusión
P=V, S=V, Q=F	$(P \wedge S \wedge \neg Q) \rightarrow A$	A = V
P=V, R=V (o Q=V)	$(P \wedge (R \vee Q)) \rightarrow E$	E = V
P=V, D=F	$(P \wedge \neg D) \rightarrow Z$	Z = V
P=V, Q=V, R=V	$(P \wedge Q \wedge R) \rightarrow Y$	Y = V
A=F, E=F, Z=F, Y=F	$(A \vee E \vee Z \vee Y) \rightarrow P$, por *modus tollens*	Si ninguna regla de ingreso se cumple, no se puede afirmar P

Ejemplo de derivación (con el formato de explicación que se guarda en accesos). Dado P=V, Q=F, R=F, S=V, D=F: $A = P \wedge S \wedge \neg Q = V \wedge V \wedge V = V$; $E = P \wedge (R \vee Q) = V \wedge (F \vee F) = F$; $Z = P \wedge \neg D = V \wedge V = V$; $Y = P \wedge Q \wedge R = V \wedge F \wedge F = F$. Se cumplen A y Z; por la precedencia Z > A, la decisión final es **retención documental**.

2.6 Clasificación de las fórmulas

Una fórmula es una **tautología** si es verdadera en todas las interpretaciones, una **contradicción** si es falsa en todas y una **contingencia** si depende de los valores. A, E, Z e Y son contingencias, lo que es deseable: ninguna aprueba ni rechaza todo. Se comprobaron además estas tautologías sobre las 32 combinaciones:

- **$(A \vee E \vee Z \vee Y) \rightarrow P$**: toda regla de ingreso exige autorización previa.
- **$Y \rightarrow E$**: una emergencia siempre implica también inspección, porque Q hace verdadera la disyunción $R \vee Q$.
- **$Y \rightarrow \neg A$**: A exige ¬Q e Y exige Q, así que nunca coinciden. Esto explica por qué $Y \wedge A$ tiene 0 casos.

3. Tablas de verdad

El simulador enumera las $2^5 = 32$ combinaciones de las cinco premisas (P, Q, R, S, D). A continuación se presentan primero las tablas de los conectivos y de cada regla por separado, y después la tabla completa con la decisión final.

3.1 Conectivos básicos

p	q	¬p	p ∧ q	p ∨ q	p → q	p ↔ q
V	V	F	V	V	V	V
V	F	F	F	V	F	F
F	V	V	F	V	V	F
F	F	V	F	F	V	V

3.2 Tablas reducidas por regla

Cada regla se reduce a sus variables relevantes. Z se muestra con Q, R y S fijas en falso, como en el análisis original; Y se cumple solo en 1 de las 8 combinaciones de (P, Q, R).

A = P ∧ S ∧ ¬Q

P	S	Q	A
V	V	V	F
V	V	F	V
V	F	V	F
V	F	F	F
F	V	V	F
F	V	F	F
F	F	V	F
F	F	F	F

E = P ∧ (R ∨ Q)

P	R	Q	E
V	V	V	V
V	V	F	V
V	F	V	V
V	F	F	F
F	V	V	F
F	V	F	F
F	F	V	F
F	F	F	F

Z = P ∧ ¬D

P	D	Z
V	V	F
V	F	V
F	V	F
F	F	F

Y = P ∧ Q ∧ R

P	Q	R	Y
V	V	V	V
V	V	F	F
V	F	V	F
V	F	F	F
F	V	V	F
F	V	F	F
F	F	V	F
F	F	F	F

3.3 Ejemplos de lectura

Estos casos muestran cómo se combinan las reglas con la precedencia (son ejemplos de lectura, no datos reales del sistema):

P	Q	R	S	D	A	E	Z	Y	Decisión	Lectura
V	F	F	V	V	V	F	F	F	Ingreso libre (A)	Solo A: ingreso libre.
V	F	F	V	F	V	F	V	F	Retención doc. (Z)	A y Z: Z gana a A.
V	F	V	V	V	V	V	F	F	Inspección (E)	A y E: E gana a A.
V	V	V	V	V	F	V	F	V	Emergencia (Y)	E e Y: Y gana a todas.
F	V	V	V	V	F	F	F	F	Denegado (¬P)	P falsa: denegado.
V	F	F	F	V	F	F	F	F	Sin regla activa	Ninguna regla activa.

3.4 Tabla de verdad completa (32 combinaciones)

Entradas: P, Q, R, S, D. Salidas: A = P∧S∧¬Q · E = P∧(R∨Q) · Z = P∧¬D · Y = P∧Q∧R. La última columna aplica la precedencia Y > Z > E > A (los colores de decisión son solo ayuda de lectura).

#	P	Q	R	S	D	A	E	Z	Y	Decisión final
1	V	V	V	V	V	F	V	F	V	Emergencia (Y)
2	V	V	V	V	F	F	V	V	V	Emergencia (Y)
3	V	V	V	F	V	F	V	F	V	Emergencia (Y)
4	V	V	V	F	F	F	V	V	V	Emergencia (Y)
5	V	V	F	V	V	F	V	F	F	Inspección (E)
6	V	V	F	V	F	F	V	V	F	Retención doc. (Z)
7	V	V	F	F	V	F	V	F	F	Inspección (E)
8	V	V	F	F	F	F	V	V	F	Retención doc. (Z)
9	V	F	V	V	V	V	V	F	F	Inspección (E)
10	V	F	V	V	F	V	V	V	F	Retención doc. (Z)
11	V	F	V	F	V	F	V	F	F	Inspección (E)
12	V	F	V	F	F	F	V	V	F	Retención doc. (Z)
13	V	F	F	V	V	V	F	F	F	Ingreso libre (A)
14	V	F	F	V	F	V	F	V	F	Retención doc. (Z)
15	V	F	F	F	V	F	F	F	F	Sin regla activa
16	V	F	F	F	F	F	F	V	F	Retención doc. (Z)
17	F	V	V	V	V	F	F	F	F	Denegado (¬P)
18	F	V	V	V	F	F	F	F	F	Denegado (¬P)
19	F	V	V	F	V	F	F	F	F	Denegado (¬P)
20	F	V	V	F	F	F	F	F	F	Denegado (¬P)
21	F	V	F	V	V	F	F	F	F	Denegado (¬P)
22	F	V	F	V	F	F	F	F	F	Denegado (¬P)
23	F	V	F	F	V	F	F	F	F	Denegado (¬P)
24	F	V	F	F	F	F	F	F	F	Denegado (¬P)
25	F	F	V	V	V	F	F	F	F	Denegado (¬P)
26	F	F	V	V	F	F	F	F	F	Denegado (¬P)
27	F	F	V	F	V	F	F	F	F	Denegado (¬P)
28	F	F	V	F	F	F	F	F	F	Denegado (¬P)
29	F	F	F	V	V	F	F	F	F	Denegado (¬P)
30	F	F	F	V	F	F	F	F	F	Denegado (¬P)
31	F	F	F	F	V	F	F	F	F	Denegado (¬P)
32	F	F	F	F	F	F	F	F	F	Denegado (¬P)

3.5 Conteos, solapamientos y distribución de decisiones

Conteos sobre las 32 combinaciones (verificados por enumeración):

Salida	Condición	Combinaciones verdaderas
A	P=V, S=V, Q=F; R y D libres	4 de 32
E	P=V y (R o Q); S y D libres	12 de 32
Z	P=V y D=F; Q, R y S libres	8 de 32
Y	P=V, Q=V y R=V; S y D libres	4 de 32

Cómo se obtienen. A: tres variables fijas (P, S, Q) y dos libres, $2^2 = 4$. E: R ∨ Q es verdadera en 3 de las 4 combinaciones de (R, Q) y S, D quedan libres: $3 \times 2^2 = 12$. Z: P y D fijas, tres libres: $2^3 = 8$. Y: P, Q y R fijas, dos libres: $2^2 = 4$.

Par de reglas	Casos	Cómo se resuelve
Z ∧ E	6	Precedencia Z > E
Y ∧ E	4	Y está contenida en E; gana Y
Z ∧ A	2	Precedencia Z > A
Z ∧ Y	2	Precedencia Y > Z
A ∧ E	2	Precedencia E > A (P=V, S=V, Q=F, R=V; D libre)
Y ∧ A	0	Mutuamente excluyentes (Y exige Q y A exige ¬Q)

Ningún par de reglas coincide en las 32 salidas, así que no hay reglas redundantes, y tampoco hay contradicciones irresolubles, porque la precedencia ordena todos los solapamientos.

Observación. El análisis original menciona cuatro solapamientos (Z∧E, Y∧E, Z∧A, Z∧Y). Al enumerar todas las combinaciones aparece además A∧E (2 casos), resuelto por la precedencia E > A. Los cuatro recuentos originales coinciden con los obtenidos aquí.

Distribución de la decisión final sobre las 32 combinaciones:

Decisión	Combinaciones
Denegado (¬P)	16 de 32
Emergencia (Y)	4 de 32
Retención documental (Z)	6 de 32
Inspección (E)	4 de 32
Ingreso libre (A)	1 de 32
Sin regla activa	1 de 32

Es decir, de las 32 combinaciones solo 1 llega a ingreso libre: el sistema es deliberadamente restrictivo.

4. Ficha del proyecto

Sistema de apoyo a la operación de una terminal portuaria desarrollado en Python + Tkinter. Aplica un LLM local (Ollama) a dos productos: un tutor académico de Inteligencia Artificial y un panel de control de acceso de unidades con clasificación de incidentes, asistente consultivo y registro de riesgos éticos.

Dato	Valor
Nombre del sistema	TransGuard
Dominio	Terminal portuaria (unidades TRK, muelles, básculas, TOS)
Interfaz	Escritorio — tkinter + ttkbootstrap (tema oscuro)
Persistencia	MongoDB (transguard) con respaldo automático a JSON local
LLM	Ollama, modelo llama3 por defecto
Idioma	Español
Ejercicios	1: tutor de IA · 2: panel integral TransGuard

5. Puesta en marcha

5.1 Requisitos

- Python 3.12+
- MongoDB en `mongodb://localhost:27017` (o Atlas vía variable de entorno)
- Ollama activo con llama3 (`ollama pull llama3`)
- Paquetes: `ollama pymongo pydantic ttkbootstrap pillow fpdf2`

5.2 Ejecución

```
:: Tutor de IA (ejercicio 1)
cd Ulises\ejercicio1_tutor_ia
python tutor_ia.py

:: TransGuard (ejercicio 2)
cd Ulises\ejercicio2_transguard
python run.py
```

5.3 Variables de entorno opcionales

Variable	Uso	Valor por defecto
MONGO_URI	Cadena de conexión	mongodb://localhost:27017
MONGO_DB	Base de datos	transguard
OLLAMA_MODEL	Modelo del clasificador/asistente	llama3

Sin MongoDB el sistema arranca igual en modo local: persiste en `datos/local_fallback.json` y lo anuncia en la barra inferior y en la pestaña Configuración. Sin Ollama el clasificador trabaja solo por reglas y el asistente muestra los registros recuperados sin redacción del modelo.

6. Diseño interno

El ejercicio 2 se separa en tres capas sin dependencia circular:

```
ui/           ← pantallas (pestañas tkinter, widgets canvas)
 │ usa
nucleo/       ← reglas, clasificador, asistente, reportes, seed
 │ usa
nucleo/db.py  ← Store: MongoDB o JSON local (misma API)
```

`ui/` nunca habla directo con pymongo ni con Ollama: todo pasa por `nucleo`. Las llamadas lentas (LLM, Mongo) corren en hilos vía `run_bg()` para no congelar la ventana.

Archivos

```
Ulises/
├─ ejercicio1_tutor_ia/
│  └─ tutor_ia.py            chat + resumen de historial
└─ ejercicio2_transguard/
   ├─ run.py                 arranque
   ├─ config.py              parámetros y rutas
   ├─ nucleo/
   │  ├─ db.py               Store/Coleccion + agregación por semana
   │  ├─ reglas.py           motor proposicional y tablas de verdad
   │  ├─ clasificador.py     reglas + LLM + fusión híbrida
   │  ├─ esquemas.py         validación pydantic
   │  ├─ asistente.py        RAG: búsqueda → contexto → respuesta citada
   │  ├─ llm.py              cliente Ollama con latencia medida
   │  ├─ reportes.py         CSV / JSON / PDF
   │  └─ seed.py             datos de demostración
   ├─ ui/                    app_tk + 10 pestañas + widgets
   ├─ datos/correos_etiquetados.json   32 correos etiquetados a mano
   └─ tests/test_nucleo.py   29 pruebas unitarias
```

7. Datos que guarda la terminal

MongoDB `transguard`, 8 colecciones:

Colección	Contenido
camiones	Padrón: camion_id (TRK-###), placa (ABC-123-D), empresa (naviera), autorizacion, certificacion_vigente, certificacion_dias_restantes
accesos	Bitácora de ingresos: premisas P Q R S D, salidas A E Z Y, decision, semaforo, explicacion (paso a paso), operador, timestamp
incidentes	remitente, asunto, cuerpo, clasificacion (categoría/prioridad/fuente/detalle), datos_extraidos, estado (nuevo / en_atencion / cerrado), requiere_revision_humana, historial[]
riesgos_eticos	modulo, descripcion, categoria, probabilidad, impacto, mitigacion, residuales, historico[]
evaluaciones_llm	run_id, modo, modelo, respuesta (exactitudes + matriz), latencia, total_correos
conversaciones	Chat del asistente: pregunta, respuesta, fuentes[], modo
configuracion	Modelo elegido, umbral de confianza, modo simulación, vigilante de turno
meta	Sellos de siembra de datos demo

Agregación (panel de control, “Incidentes por semana”): `$group` por `clasificacion.categoria` + `$isoWeek`/`$isoWeekYear` sobre `timestamp`. En modo local la misma agrupación se calcula con `datetime.isocalendar()`.

8. Motor de decisiones (lógica proposicional)

> El desarrollo formal de la lógica proposicional y las tablas de verdad completas (32 combinaciones) están en las secciones 2 y 3. Esta sección conserva la descripción de la implementación del motor.

8.1 Premisas del dominio portuario

Premisa	Significado
P	Unidad con autorización previa de ingreso
Q	El peso rebasa el límite de la terminal
R	Mercancía catalogada como peligrosa
S	Operador con licencia/certificado vigente
D	Documentación portuaria completa (BL, manifiesto)

8.2 Reglas

```
A = P ∧ S ∧ ¬Q       ingreso libre
E = P ∧ (R ∨ Q)      canal de inspección
Z = P ∧ ¬D           retención documental (nueva)
Y = P ∧ Q ∧ R        protocolo de emergencia (nueva)
```

Justificación de las reglas nuevas

- **Z:** una unidad autorizada que llega sin el BL o el manifiesto no puede desconsolidar; se retiene en el andén de revisión documental hasta regularizar, aunque las demás reglas la dejaran pasar.
- **Y:** sobrepeso y mercancía peligrosa juntos multiplican el riesgo (falla estructural + posible derrame); bloquea el acceso hasta permiso especial.

Precedencia: Y > Z > E > A. Si P es falso no hay vía de ingreso: denegado.

8.3 Tablas de verdad de las reglas nuevas

Z = P∧¬D (Q, R, S fijas en falso):

P	D	Z
V	V	F
V	F	V
F	V	F
F	F	F

Y = P∧Q∧R se cumple en 1 de las 8 combinaciones de (P, Q, R): solo cuando las tres son verdaderas.

8.4 Trazabilidad y análisis

Cada evaluación guarda en `accesos` la explicación paso a paso (P=V (…), A = P∧S∧¬Q = V∧V∧V = V, regla que disparó la decisión final).

Análisis exhaustivo de las 32 combinaciones (pestaña Simulador):

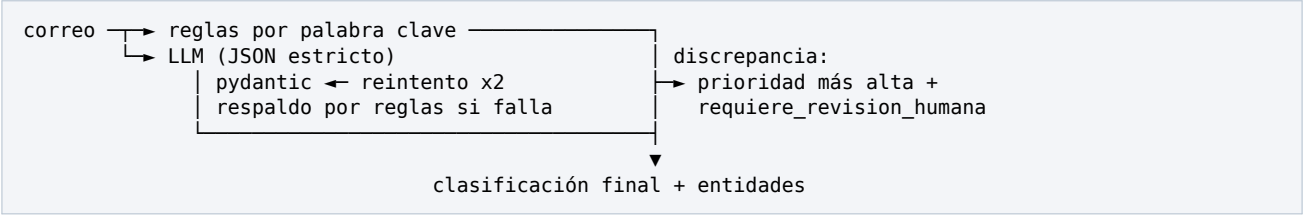
- **Solapamientos:** Z∧E en 6 casos, Y∧E en 4, Z∧A en 2, Z∧Y en 2 — todos resueltos por la precedencia.
- **Redundantes:** ninguna (ningún par de reglas coincide en las 32 salidas).
- **Contradicciones irresolubles:** ninguna.

9. Clasificador mixto de incidentes

9.1 Categorías del dominio

`carga_peligrosa · peso_excedido · acceso_indebido · falla_equipo · falla_sistema · estado_conductor · otro`

9.2 Flujo



Entidades extraídas: placa (ABC-123-D), camion_id (TRK-###), peso_reportado_kg, ubicacion (muelle/andén/grúa/dársena…).

9.3 Prompt del clasificador (texto íntegro)

```
Eres el clasificador de incidentes de la terminal portuaria TransGuard.
Analiza el correo y contesta EXCLUSIVAMENTE con un objeto JSON válido (sin
markdown ni texto extra) con esta estructura exacta:
{
  "categoria": "<carga_peligrosa | peso_excedido | acceso_indebido |
                falla_equipo | falla_sistema | estado_conductor | otro>",
  "prioridad": "<baja | media | alta | critica>",
  "entidades": {
    "placa": "<placa tipo ABC-123-D o null>",
    "camion_id": "<id tipo TRK-### o null>",
    "peso_reportado_kg": <numero en kg o null>,
    "ubicacion": "<lugar citado o null>"
  },
  "resumen": "<frase corta del incidente>"
}
Guía:
- carga_peligrosa: derrames, fugas, químicos, inflamables, gas, solventes.
- peso_excedido: báscula por encima del límite, sobrecarga.
- acceso_indebido: personas o unidades sin autorización, vallas forzadas.
- falla_equipo: grúas, cámaras, lectores, básculas dañadas o apagadas.
- falla_sistema: TOS, aplicaciones, pantallas, errores de software.
- estado_conductor: fatiga, somnolencia, conductor en mal estado.
- prioridad critica: peligro inmediato; alta: seguridad comprometida;
  media: operación afectada; baja: lo demás.
- Copia solo datos presentes en el correo; si falta un dato usa null.
```

9.4 Experimento sobre el dataset (32 correos etiquetados a mano)

Ejecutado desde la pestaña Evaluación, modo reglas:

Métrica	Resultado
Exactitud en categoría	90.6 % (29/32)
Exactitud en prioridad	75 % (24/32)
Latencia media	< 1 ms (sin LLM)

Matriz de confusión (esperada → obtenida):

Esperada	Obtenida	Casos	Nota
carga_peligrosa	carga_peligrosa	4	

Esperada	Obtenida	Casos	Nota
peso_excedido	peso_excedido	5	
acceso_indebido	acceso_indebido	4	
falla_equipo	falla_equipo	5	
falla_sistema	falla_sistema	5	
estado_conductor	estado_conductor	4	
otro	otro	2	
estado_conductor	otro	1	correo 24: “aliento alcohólico”
carga_peligrosa	otro	1	correo 25: “kemiko/corosibo” con faltas
acceso_indebido	otro	1	correo 27: redacción muy informal

Hallazgo ético real: los 3 errores de categoría corresponden a correos con ortografía informal o vocabulario no previsto — evidencia directa del riesgo de sesgo documentado en la matriz de riesgos. El modo LLM e híbrido se evalúan desde la misma pestaña (llama3 tarda ~90 s por correo en CPU) y cada corrida queda guardada en `evaluaciones_llm` para comparar.

10. Asistente consultivo (RAG)

El chat de la pestaña Asistente nunca contesta de memoria:

1. Extrae referencias de la pregunta (TRK-###, placas ABC-123-D, palabras como incidente / acceso / riesgo).
2. Consulta camiones, accesos, incidentes o riesgos_eticos según la intención.
3. Construye un contexto de máximo 8 registros reales.
4. El LLM responde solo con ese contexto, citando [coleccion#id].
5. Sin registros → respuesta fija: “No tengo información registrada sobre eso.”

Prompt del asistente (texto íntegro)

```
Eres el asistente de la terminal portuaria TransGuard.
Contesta la pregunta del operador usando UNICAMENTE la información del
CONTEXTO (registros reales de la base de datos).
Reglas:
1. Cita el registro de origen con su etiqueta, p. ej. [accesos#abc123].
2. Si el contexto NO contiene la respuesta, responde exactamente:
   "No tengo información registrada sobre eso." No inventes nada.
3. Español claro y breve (máximo 3 párrafos cortos o viñetas).
4. Al explicar una decisión de acceso menciona las premisas (P,Q,R,S,D)
   que la provocaron según el razonamiento guardado.
```

Verificado: la pregunta “¿Por qué TRK-201 fue enviado a inspección?” recupera la ficha del camión y sus últimos accesos, y cita [camiones#…] / [accesos#…]. Si Ollama está caído el chat no muere: muestra los registros recuperados sin redacción (modo: reglas).

11. Registro de riesgos éticos

6 riesgos propios del sistema, cargados por la siembra y editables desde la GUI con matriz 5×5 inherente y residual:

Módulo	Riesgo	P×I	Inherente	Residual
Clasificador mixto	Fabricar categorías o datos inexistentes ante texto ambiguo	4×4	16 alto	6 medio

Módulo	Riesgo	P×I	Inherente	Residual
Motor de palabras	Subclasificación de correos con ortografía informal / jerga portuaria	4×3	12 alto	6 medio
Expediente de conductores	Datos personales de operadores (certificados, fatiga) en la base	3×5	15 alto	8 medio
Operación asistida	Dejar de contrastar decisiones automáticas por costumbre	3×4	12 alto	6 medio
Asistente contextual	Inventar respuestas plausibles sin registros suficientes	3×4	12 alto	6 medio
Verificador documental	Frenar embarques legítimos por umbrales mal ajustados	2×4	8 medio	3 bajo

Mitigaciones aplicadas en el código: pydantic estricto + reintentos + respaldo, bandera `requiere_revision_humana`, dataset con lenguaje informal, RAG con citas obligatorias, explicación visible de cada decisión y simulador para validar cambios de reglas antes de aplicarlos.

12. Pantalla de escritorio (ejercicio 2)

10 pestañas del notebook, tema oscuro superhero, cabecera con estado de MongoDB y del LLM, barra inferior con motor de persistencia y tamaño del dataset:

Pestaña	Qué hace
Panel	KPIs (unidades atendidas, incidentes abiertos, riesgos críticos, padrón), filtros por fecha, 3 gráficas en canvas, últimos movimientos
Acceso	Búsqueda por placa o TRK, premisas con casillas, semáforo, explicación paso a paso, bitácora filtrable
Simulador	Interruptores en vivo + tabla de verdad de A/E + tablas de Z e Y + análisis de solapamientos
Incidentes	Pegar correo → clasificar (reglas/LLM/híbrido), corregir resultado, cambiar estado, historial, filtro de revisión humana
Evaluación	Experimento sobre el dataset con barra de progreso, exactitudes, matriz de confusión, detalle por correo, histórico de corridas
Asistente	Chat RAG con fuentes citadas y sugerencias de preguntas
Riesgos	Matriz 5×5 inherente y residual + CRUD con validación pydantic
Unidades	Padrón de camiones: alta/edición/baja con validación de formato
Reportes	Vista previa + exportación CSV / JSON / PDF por colección
Configuración	Modelo Ollama, umbral de confianza, modo simulación, vigilante de turno, reconexión a MongoDB, resembrado demo

El ejercicio 1 es una ventana única: cabecera con estado de Ollama y selector de modelo, área de chat con mensajes alineados, entrada con Enter, botón **Resumen del historial** (viñetas en segunda persona generadas por el propio LLM) y reinicio de conversación.

System prompt del tutor (texto íntegro)

```
Actúa como docente universitario experto en Inteligencia Artificial.
Tu trabajo es apoyar a alumnos de licenciatura en sus dudas.
Indicaciones:
1. Responde en español y de forma clara.
2. Apóyate en ejemplos cotidianos.
3. Detalla los procedimientos uno por uno.
4. Baja el nivel técnico si el alumno va empezando.
5. Incluye fragmentos en Python cuando ayuden a entender.
6. Si el alumno se equivoca, muéstrale el error y cómo corregirlo.
7. No entregues solo la solución: expón también el porqué.
8. Termina las explicaciones largas con una idea resumen.
```

13. Cómo se comprobó

Prueba	Resultado
python -m unittest discover -s tests	29/29 OK (reglas, clasificador, esquemas)
Apertura de la app TransGuard	las 10 pestañas construyen y refrescan sin error
Apertura del tutor	ventana, chat y estado de Ollama OK
Seed en MongoDB	8 unidades, 17 accesos, 14 incidentes, 6 riesgos
Agregación categoría/semana	$group+$isoWeek con resultados por semana
Asistente RAG	cita registros reales; “sin información” cuando no hay datos
Evaluación por reglas	90.6 % categoría / 75 % prioridad
Exportación	CSV y JSON íntegros; PDF generado (fpdf2)
Modo degradado	sin MongoDB persiste en local_fallback.json; sin Ollama clasifica por reglas

Ejecutar las pruebas:

```
cd Ulises\ejercicio2_transguard
python -m unittest discover -s tests -v
```

14. Limitaciones conocidas

- llama3 en CPU tarda ~90 s por clasificación; la evaluación completa en modo LLM/híbrido puede durar ~1 h sobre los 32 correos (el progreso se muestra en pantalla y puede lanzarse por separado por modo).
- El extractor de peso interpreta “toneladas/tonelada/ton/kg”; otras unidades no se capturan.
- El modo local es un respaldo de emergencia, no un reemplazo: la agregación por semana se calcula en memoria y no hay índices.
- El tutor no persiste el historial entre sesiones (vive en memoria de la ventana, a propósito, para simplificar el ejercicio).
- El análisis PEAS (sección 1) es un análisis de diseño y debe contrastarse con el comportamiento observado del sistema.

15. Solución rápida de problemas

Síntoma	Causa probable	Acción
“LLM no disponible” en cabecera	Ollama apagado	`ollama serve` y revisar modelo en Configuración
“Respaldo local” en barra	Mongo apagado	iniciar el servicio o botón Reintentar conexión
Clasificación lenta	llama3 en CPU	normal; usar modo reglas para pruebas rápidas
Ventana en blanco al arrancar	ttkbootstrap ausente	`pip install ttkbootstrap` (sin él funciona con ttk estándar)
Exportar PDF falla	falta fpdf2	`pip install fpdf2`